FINTECHATHON 2026 · SHENZHEN · INTERNATIONAL TRACK

ClauseGuard

TOPIC D · AI INSURANCE ASSISTANT

Technical Documentation.

Ask in your own words. Get the clause.

System architecture, AI agent design, core algorithms, security design, deployment and testing.



33/33 claim tasks passed

0/27 attacks succeeded

PRODUCT	ClauseGuard, an AI insurance assistant that decides on clauses, not guesses
TEAM	Bunga Laelatul Muna (Team Lead) · Izra Noor Zahara Aliya · Wan Sabrina Mayzura
INSTITUTION	Institut Teknologi Sepuluh Nopember (ITS), Surabaya, Indonesia
LIVE APP	clauseguard.up.railway.app
REPOSITORY	github.com/Bungalm/CLAUSEGUARD
VERSION	October 2026

Contents.

Nine sections, from the problem to the references.

01 Problem & Scope 03

1.1 Problem 03
1.2 Existing solutions 03
1.3 Proposed solution 03
1.4 Comparison with existing products 04
1.5 Prior research methods, 2020 to 2026 05
1.6 Research gap and contribution 06
1.7 Demo accounts 07

02 Features 08

03 System Architecture 11

3.1 System components 11
3.2 Claim flow 12
3.3 Decision flow within one turn 12
3.4 Technology stack and deployment 13
3.5 API 13
3.6 Third-party services 14
3.7 Database 15
3.8 Sandbox insurer 15
3.9 Environment configuration and judge access 16
3.10 Project structure 17

04 Core AI Agent 19

4.1 AI agents and prompt design 19
4.2 Planning and tool calling 20
4.3 Memory 20
4.4 Retrieval 20
4.5 Claim allocation model 20
4.6 Claim risk assessment 20
4.7 Guardrails and explainability 21

05 Security Design 22

5.1 Permission tiers 22
5.2 Risk controls 22

06 Innovation & Interaction 24

6.1 Difference from existing solutions 24
6.2 Interaction design 24
6.3 Interface design and visual hierarchy 24

07 Testing & Results 26

08 Limitations & Known Risks 27

09 References 28

CONTENTS

01 Problem & Scope

Why policyholders lose benefits they are already entitled to.

1.1 Problem

Individuals and households increasingly hold multiple insurance policies from different providers through employers, digital platforms and public programmes. In Indonesia, a household may hold BPJS Kesehatan coverage, employer-sponsored insurance, private health insurance and accident cover at the same time. Each policy specifies its own waiting periods, claim deadlines, documentation requirements, benefit limits and payment order. These provisions are spread across lengthy policy documents, which makes it difficult for policyholders to understand their coverage, calculate eligible benefits and meet claim requirements.

This complexity creates risks across the insurance lifecycle. Policyholders may miss claim deadlines, submit claims during waiting periods, approach insurers in the wrong payment order, under-use overlapping or stackable benefits, or let policies lapse by missing premiums during grace periods. In the United States, KFF reported that roughly one in five in-network health insurance claims was denied in 2023, while fewer than 1% of denied claims were appealed [1]. In Indonesia, the Financial Services Authority (Otoritas Jasa Keuangan, OJK) identified claim-related difficulties as the most frequent category of insurance complaints in 2023 [2].

1 in 5
in-network health claims denied in the US in 2023 [1]

< 1%
of denied claims were appealed [1]

#1
claim difficulty is the top insurance complaint at OJK [2]

1.2 Existing solutions

Insurers and insurtech companies have introduced virtual assistants and digital claim channels to improve service [3]–[6]. Lemonade describes its platform as built around AI from day one [7], and Indonesian platforms such as PasarPolis, Qoala and Lifepal focus on distribution and digital purchase [8]. Researchers have also proposed a benchmark for identifying gaps and overlaps in insurance contracts [9]. These tools answer questions or route claims, but policyholders still lack a reliable way to determine their entitlements directly from the policy wording, across several policies at once, and to verify the clause behind each determination.

1.3 Proposed solution

ClauseGuard is an AI-powered insurance assistant that analyses policy and claim documents to help policyholders understand their coverage and manage claims. It identifies applicable benefits, calculates eligible amounts, tracks filing deadlines, checks documentation requirements and determines the payment order across policies. Every determination is linked to the policy clause that supports it. ClauseGuard also flags claim-readiness issues and prepares the insurer's claim form for

review. Claims are submitted only after explicit confirmation by the policyholder, and every agent action is recorded in an audit trail.

1.4 Comparison with existing products

Table 1 compares ClauseGuard with four products whose use of AI is documented in public sources. The logos belong to their owners and identify the products compared; n/d means not stated in the sources we checked.

Table 1 Comparison with publicly documented AI insurance products

CAPABILITY	ClauseGuard This work	Travelers AI Claim Assistant 2026 [10]	Allianz Insurance Copilot 2024–2025 [11]	Lemonade "AI Jim" 2023 [12]	Qoala smartphone video claims 2020 [13]
Primary user	Policyholders; insurer claims desk (sandbox)	Policyholders filing auto property-damage claims	Claims handlers	Policyholders	Policyholders
Role of AI	Language understanding and drafting; decisions made by the rules engine	Voice assistant that answers policy questions, collects claim details and submits claims	Summarises claim and contract data, compares claims with policy terms and flags factors a handler might miss	Chatbot that reviews the claim, checks policy conditions and triggers payment	Assesses smartphone damage from a submitted video
Interprets policy wording into executable rules	Yes	n/d	Partial compares claims against policy terms	Partial checks policy conditions	No
Cites the governing clause for each answer	Yes	n/d	n/d	n/d	No
Coordinates payment across multiple policies	Yes	n/d	n/d	n/d	No
Pre-submission readiness check against policy rules	Yes	n/d	n/d flags factors for the handler	n/d	No
Decides and pays claims without a human	No the insurer officer decides	n/d decision authority not stated	n/d the tool supports handlers	Yes for claims the chatbot approves (reported)	Partial eligibility result within minutes
Policyholder confirms before submission	Yes one tap on the Confirm card	n/d	Not applicable used by handlers	n/d	n/d
Permission tiers and audit log for agent actions	Yes	n/d	n/d	n/d	n/d

CAPABILITY	ClauseGuard This work	Travelers AI Claim Assistant 2026 [10]	Allianz Insurance Copilot 2024–2025 [11]	Lemonade "AI Jim" 2023 [12]	Qoala smartphone video claims 2020 [13]
Reported outcome	**84.6% coverage-decision accuracy on 390 public benchmark labels (small test set)**	85–90% of customers complete filing through the AI (company-reported)	No figures found in the sources checked	Close to half of claims handled with the technology (company-reported)	None stated
Scope	**Life, health, accident, critical illness and public coordination programmes (case study)**	Auto property damage	Motor claims in Austria; property planned	Not stated	Smartphone damage

1.5 Prior research methods, 2020 to 2026

Table 2 compares ClauseGuard with research on insurance question answering, contract analysis, coverage assessment and secure AI agents. The review also includes counterfactual explanations [20] and CaMeL [19], because they inform the system's explanation and security design.

Table 2 Methodological comparison with prior work

WORK	TASK AND DATA	ARCHITECTURE	DECISION AUTHORITY AND EVIDENCE	REPORTED EVALUATION
Na et al. (2022) [14]	Single-turn Korean cancer insurance questions: 2,295 filtered pairs, 892 manually created question-answer pairs and 817 FAQ pairs	KoELECTRA for intent detection and slot filling, with Sentence-BERT for FAQ retrieval	Answers grounded in FAQ entries	Intent micro-F1 0.71, slot micro-F1 0.95, FAQ accuracy 70%
InsQABench (2025) [15]	Chinese insurance questions over structured databases and documents	SQL-ReAct for structured data and RAG-ReAct for documents	Answers grounded in retrieved text	Benchmark reported without numeric scores in the reviewed text
INSEva (2025) [16]	38,704 Chinese examples from examinations, regulations and business data	Benchmark evaluating eight LLMs with model-assessed faithfulness and completeness	Evaluation benchmark, not a deployed system	Best aggregate score 82.74; logical reasoning weakest, best 73.66
Beauchemin and Khoury (2026) [17]	Quebec property and casualty insurance: 807 multiple-choice questions from examination handbooks	Closed-book LLMs compared with RAG using five retrieved chunks and context compression	Answers grounded in retrieved chunks	Best accuracy 78.68% with RAG; one model fell from 74.65% to 14.06%

WORK	TASK AND DATA	ARCHITECTURE	DECISION AUTHORITY AND EVIDENCE	REPORTED EVALUATION
Pokharel et al. (2026) [18]	CPT code coverage under plan documents: 1.84 million labelled pairs and 814 evaluation codes	Fine-tuned Longformer retrieval, GPT-4.1–generated Boolean attributes and rules, and a PyKnow rule engine	Advisory outputs for human reviewers, with auditable rule rationales	Accuracy 0.87, F1 0.93 vs 0.82 and 0.89 for GPT-4.1 on full documents; about $22 vs $38,720 for ~11,000 codes
Mridul et al. (2026) [9]	Life insurance: ten contracts and 58 scenarios	Ontology and knowledge base queried with SPARQL, against a text-only LLM baseline	Scenario outcomes supported by clause-level evidence	Improved consistency and diagnosis over the baseline; no numeric metrics in the abstract
Debenedetti et al. (2025) [19]	General agent tasks on the AgentDojo benchmark	Separates control flow from data flow, attaches capabilities to data and checks policies at each tool call	Security-focused agent framework	77% of tasks solved with provable security vs 84% undefended
ClauseGuard This work	Policy-to-rule compilation, synthetic personas and a sandbox insurer	Five specialised LLM agents for language tasks, a deterministic rules engine, a Guardian tier gate and typed handles	The insurer officer decides claims; the rules engine computes coverage and payment; outputs cite clauses	Coverage-decision accuracy 84.6% on 390 labels; 0 of 27 attacks succeeded

1.6 Research gap and contribution

The reviewed studies provide foundations for insurance question answering, policy document retrieval, rule-based coverage assessment, clause-level explanations and secure agent design. ClauseGuard integrates these approaches into one workflow for managing claims across multiple policies. Its main contributions are:

1. **Deterministic coverage and benefit calculation.** A rules engine computes coverage, benefit amounts, deadlines and payment order from compiled policy rules, while language models handle interpretation and drafting. Pokharel et al. [18] also combine language models with a rule engine, focused on CPT code coverage and human review.
2. **Coordination across multiple policies.** ClauseGuard determines the payment order for an event covered by several policies, extending the scope from interpreting one contract to coordinating benefits across a policyholder's portfolio.
3. **Controlled agent actions.** Typed handles restrict the inputs accepted by higher-permission tools, including claim submission. This adapts the control-flow and data-flow separation of CaMeL [19] to insurance claim processing.
4. **Claim readiness assessment.** ClauseGuard checks claim information against the applicable policy requirements and flags missing or correctable items before submission, and searches for the smallest legitimate change that resolves a readiness issue.
5. **Clause-level evidence and auditable execution.** Each coverage determination links to its governing clause, and a hash chain records system actions for auditability and tamper detection.

1.7 Demo accounts

The project uses synthetic personas to simulate the claim process across representative cases. Each account loads its own policies and claims in the sandbox insurer.

Table 3 lists the three demo accounts, their profiles and the policies each one holds.

Table 3 Demo accounts and their policy portfolios

PERSONA	PROFILE	POLICIES
Rizky, 24	Postgraduate student handling an inpatient claim for dengue fever	Sompo hospital cash plan, term life insurance with an overdue premium, critical illness cover, BPJS Kesehatan class 2
Santoso family	Family of four; the head of household is a private-sector employee	Group health and life insurance, family hospital cash plan, the wife's private health insurance, BPJS Kesehatan class 1
Dewi, 29	Online ride-hailing courier	Personal accident insurance, the platform's on-duty accident cover, BPJS Kesehatan class 3 with three months in arrears, Jasa Raharja

02 Features

Three core features and three supporting features.

Table 4 summarises the three core features and the financial task each one completes.

Table 4 Core features

FEATURE	PURPOSE	FINANCIAL TASK COMPLETED
ClauseLens	Understand the customer's cover	Decides whether an event is covered, how much the benefit is, the filing deadline and which policy pays first, with the clause quoted
ClaimFlow	Handle the customer's claim	Reads hospital documents, matches them to the required list, fills the insurer's own claim form, submits after confirmation and relays the insurer's decision
TrustLock	Keep control with the customer	Limits every agent action with four permission tiers, single-use confirmations and an audit log

The three core features are shown in the app below. **Figure 1** shows a ClauseLens verdict with the clause that decides it, **Figure 2** shows the ClaimFlow claim form the customer reviews before anything is sent, and **Figure 3** shows the TrustLock autonomy switch and the Guardian permission tiers.

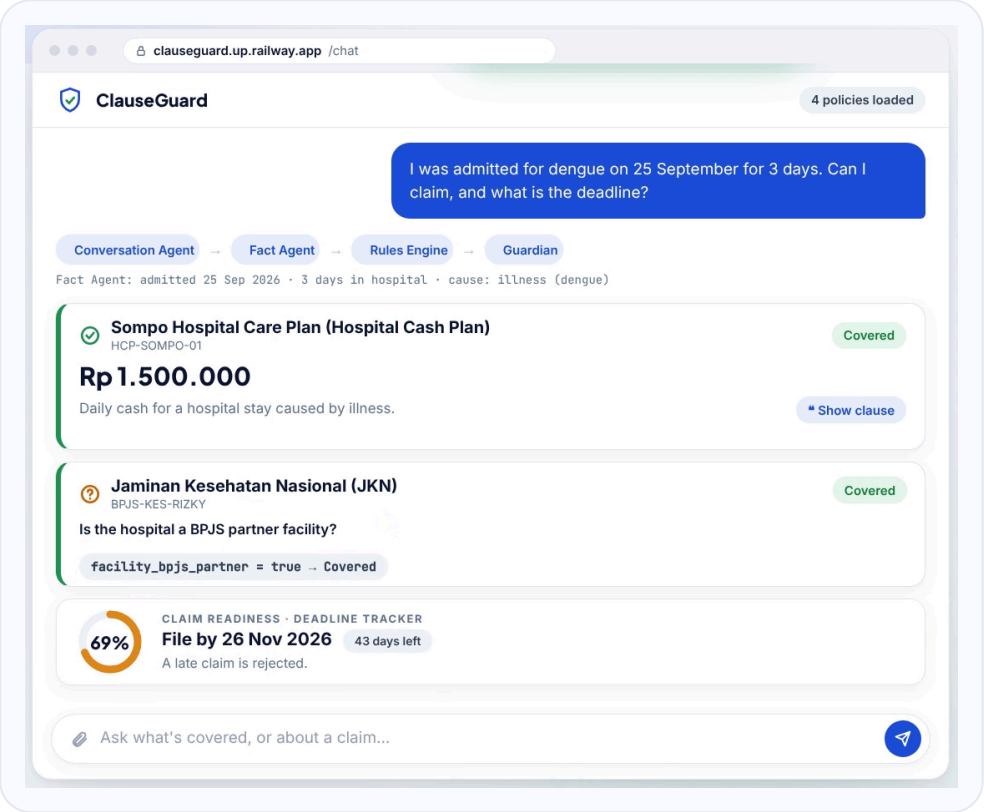


Figure 1 ClauseLens: a verdict with the deciding clause

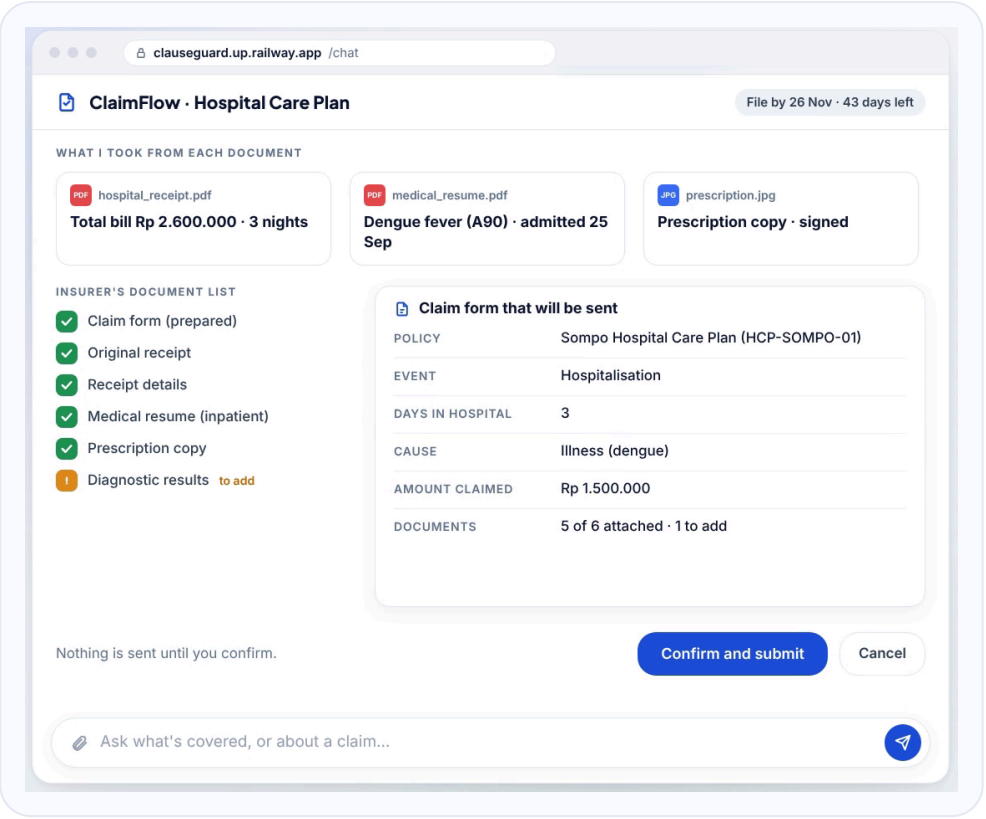


Figure 2 ClaimFlow: the claim form before it is sent

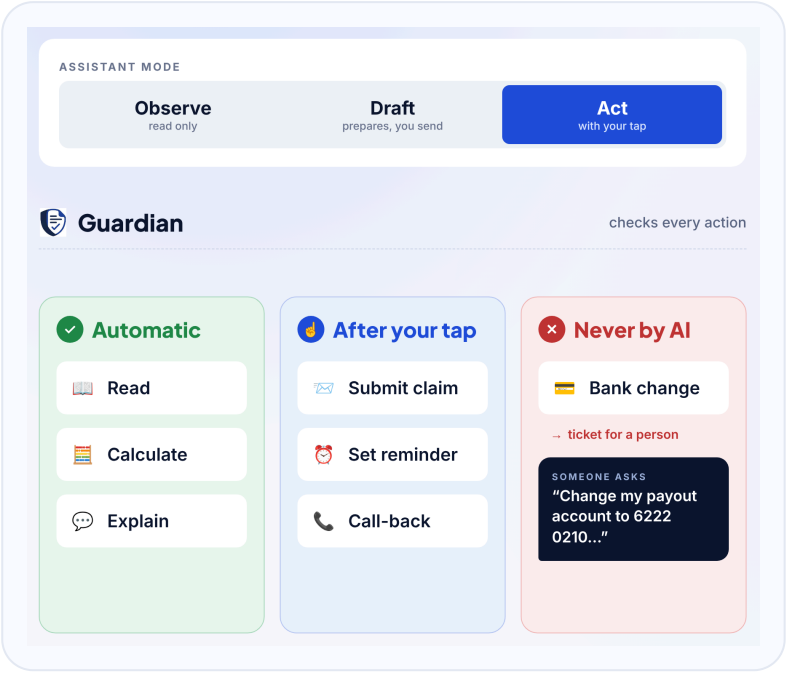


Figure 3 TrustLock: autonomy mode and Guardian permission tiers

Table 5 lists the three supporting features that complement the core flow.

Table 5 Supporting features

FEATURE	PURPOSE
Claim Simulator	Answers what-if questions such as “what if I stay five days?” without touching the real claim
Deadline Tracker	Reminds the customer of claim deadlines and premium grace periods before they pass
PDF Export	Saves the answer, clause evidence and claim form as a document

System Architecture

03 AI agents understand and explain. The rules engine decides. Guardian enforces permissions.

3.1 System components

Figure 4 shows the components of ClauseGuard. AI agents understand messages and documents and explain results. The rules engine makes every decision. Guardian enforces permissions on every tool call. Every step is written to the audit log.

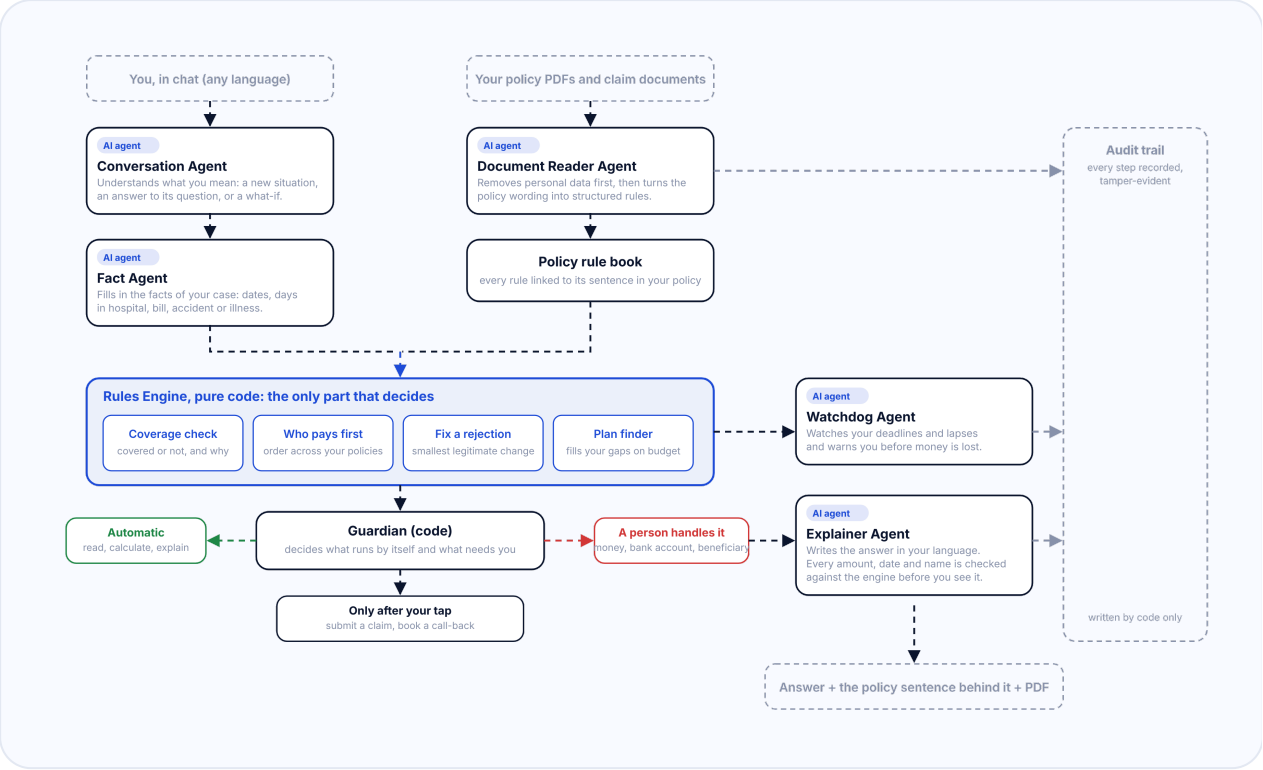


Figure 4 ClauseGuard component architecture

Table 6 describes the role of each component shown in the architecture.

Table 6 Role of each component

COMPONENT	ROLE
UI and dialogue	Customer web app (chat, verdict cards, claim form, document upload) and a claims desk for insurer officers
Orchestrator	Agent loop that runs planning, permission checks, tool calls, the rules engine and answer composition on every turn
LLM	Google Gemini for understanding messages, reading documents and writing sentences
AI agents	Five agents with narrow tasks, described in Section 4.1
Tools and API	19 permission-tiered tools that call the sandbox insurer API

COMPONENT	ROLE
Rules engine	Deterministic code that decides coverage, amount, deadline, payer order and claim readiness
Guardian	Policy enforcement point that checks every tool call
Data storage	Postgres on Supabase for the audit log, chat history, claims and tickets
Sandbox insurer	Simulated insurer with a claims desk where officers decide claims

3.2 Claim flow

Figure 5 follows one claim from the first question to the insurer's decision. The colour of each box shows who works at that step.

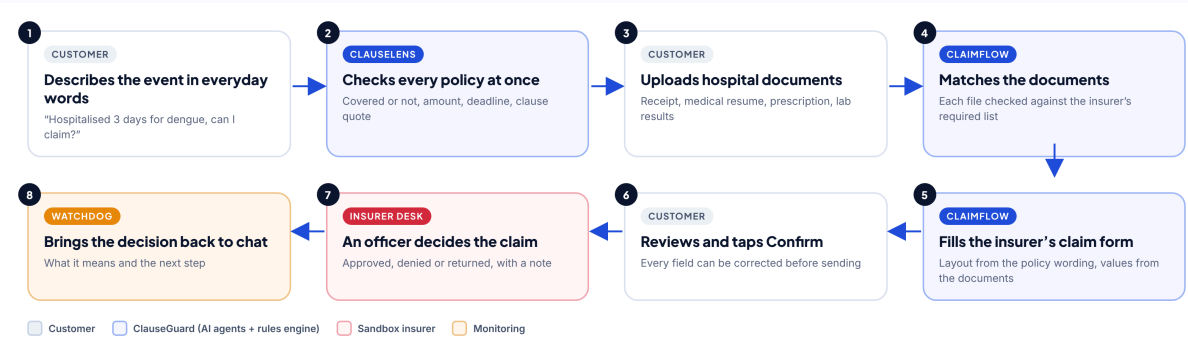


Figure 5 Claim flow from question to decision

3.3 Decision flow within one turn

Figure 6 shows what happens each time the customer sends a message. Safety rules are decided by code before any model reads the intent of the message, and the verdict is always computed by the rules engine.

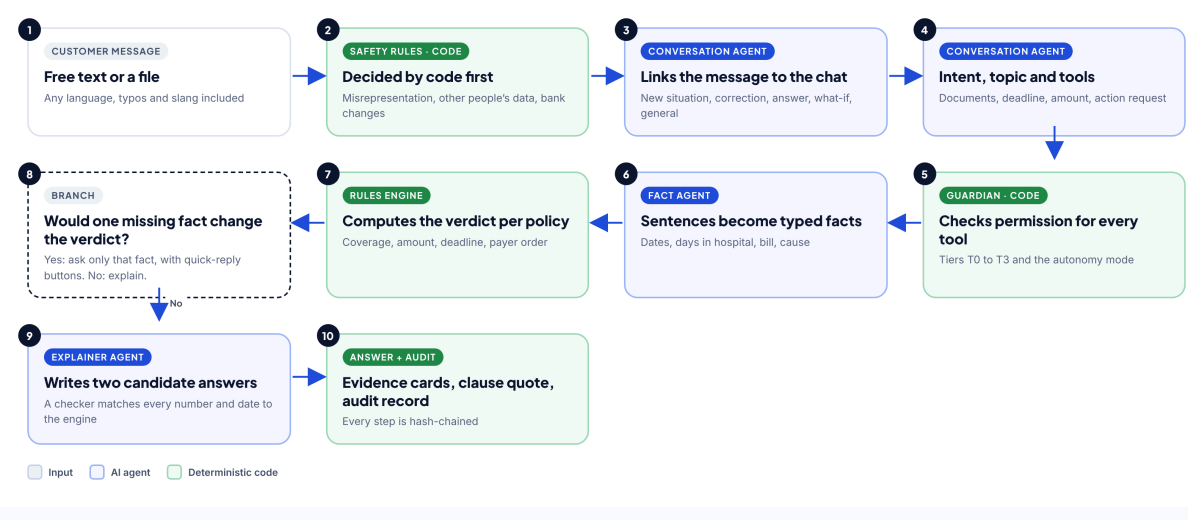


Figure 6 Decision flow within one conversation turn

3.4 Technology stack and deployment

Each technology was chosen because it supports structured, safe and inspectable output.

Table 7 lists each layer of the stack, the technology used and why it was chosen.

Table 7 Technology stack and rationale

LAYER	TECHNOLOGY	RATIONALE
Rules engine and agents	Python · FastAPI · Pydantic	Strictly typed schemas for policy rules, facts and model output
LLM	Google Gemini Flash Lite	Schema-constrained JSON output, fast, low cost and fluent in everyday Indonesian
Optimisation	SciPy (MILP · HiGHS)	Gap-cover product selection with optimal, checkable results
Web app	Next.js · TypeScript · Tailwind CSS	Chat interface with interactive cards for customers and the insurer desk
Accounts and storage	Supabase Auth and Postgres	Confirmed email login, row-level security and a persistent audit log
Hosting	Railway · Southeast Asia region	Separate web and API services close to the database, redeployed automatically from the repository

3.5 API

The ClauseGuard sandbox exposes 52 JSON endpoints built with FastAPI. The customer's browser calls them through the /api/cg route of the web app, which forwards requests to the API. Customer endpoints require an X-Session-Token header obtained from POST /login; insurer-desk endpoints require an X-Admin-Token header. Errors are returned with an HTTP status code and a detail body. **Table 8** groups the endpoints.

Table 8 API endpoint groups

GROUP	ENDPOINTS	ACCESS
Account and session	`POST /login` · `GET /session` · `POST /session/reset` · `POST /session/autonomy` · `POST /session/clock` · `POST and DELETE /session/api-key` · `GET /personas` · `GET /health` · `GET /system/status`	/login, /personas, /health and /system/status are open; the rest need a session
Conversation	`POST /chat` · `POST /chat/confirm` · `GET /chat/pending` · `POST /confirm` · `POST /form`	Session
Policies and documents	`GET /policies` · `GET /portfolio` · `POST /policies/upload` · `POST /upload` · `GET /uploads/{doc_id}`	Session

GROUP	ENDPOINTS	ACCESS
Entitlement analysis	`GET /entitlement/plans` `/allocation` `/timeline` `/digest` `/twin` `/recourse` `POST /entitlement/whatif` `POST /export/pdf`	Session
Claims and history	`GET /claims/{claim_id}` `GET /history` `GET` `PUT` `PATCH and DELETE /history/{conversation_id}` `DELETE /history`	Session
Agent tools and security	`GET /tools` `POST /tools/{name}` `GET /audit` `GET /audit/verify` `GET /security/status` `GET /demo/attacks` `POST /demo/attack` `POST /demo/audit_tamper`	/tools and /audit/verify are open; the rest need a session
Insurer desk and admin	`GET /backoffice/claims` `GET /backoffice/claims/{claim_id}` `POST /backoffice/claims/{claim_id}/decide` `GET /backoffice/documents/{doc_id}` `GET /backoffice/tickets` `GET` `POST and DELETE /admin/kill` `POST /system/add-key`	Admin token

Every endpoint that touches customer data looks up the session first and filters data by that account. A request that names a claim belonging to another account is logged as IDOR_ATTEMPT in the audit log.

3.6 Third-party services

ClauseGuard relies on a small set of third-party services and public data. **Table 9** lists what each one is used for and what data it receives.

Table 9 Third-party services and data sources

SERVICE	USED FOR	DATA SENT	NOTES
Google Gemini API	Reading policies and claim documents, understanding messages, writing answers, embedding diagnosis codes	Customer messages and document text with personal data removed	Several keys rotate when a quota runs out; customers can use their own key
Supabase Auth	Login with a confirmed email	Account email and password	The backend verifies the account token on every login
Supabase Postgres	Audit log, chat history, claims and tickets	Application data owned by the account	Row-level security on with no public policy; the backend uses a dedicated connection
Railway, Southeast Asia	Running the web and API services	All application traffic	Redeployed automatically from the repository

SERVICE	USED FOR	DATA SENT	NOTES
Google Fonts via Next.js	Inter and Sora typefaces	No customer data	Downloaded at build time
ICD-10-CM code list (CMS, NCHS)	Basis for diagnosis-code search	None	Public domain, stored in the repository
multilingual-e5-small (ONNX)	Local diagnosis-code search	None	Optional, runs on CPU
KFF, CMS marketplace data, arXiv 2604.10853 benchmark	Evaluation and plan selection	None	Offline testing only

3.7 Database

The backend selects its storage through the CG_STORAGE variable. Local mode uses SQLite and JSONL files for development and testing; postgres mode uses Supabase Postgres and runs in the hosted service.

Table 10 groups the database tables by function.

Table 10 Database tables by function

AREA	TABLES	FUNCTION
Evidence and history (Postgres)	`audit_events` `audit_head` `conversations` `claims_log` `tickets_log` `claim_payments_log` `schema_migrations`	Hash-chained audit log, chat history, claims and tickets written by agents or the insurer desk, schema version
Sandbox insurer (SQLite, reseeded on start)	`customers` `policies` `coverages` `riders` `exclusions` `claim_rules` `other_insurance_clauses` `claims` `claim_payments` `payments` `beneficiaries` `documents` `facilities` `tickets` `audit_log`	The three personas, policies and benefits, claims, premium payments, documents, partner hospitals and tickets
Tariff reference	`tariffs_ina_cbg`	Reference tariffs in the sandbox; not yet used by the rules engine for INA-CBG grouping
Clause store	`clause_policies` `clause_tuples` `clause_embeddings` `clause_owners`	Extracted policy clauses and their owners

Every Postgres table enables row-level security with no policies, and all anon and authenticated grants are revoked, so the public key cannot read any data. The audit_events table also has a trigger that rejects UPDATE and DELETE. User sessions are held in the memory of a single process; after a server restart the web app signs in again automatically. A shared session store for multiple instances is on the roadmap.

3.8 Sandbox insurer

The sandbox is a simulated insurer that stands in for a real insurer system. It holds the three personas, their policies and benefits, partner hospitals and claim intake. Insurer officers work at the claims desk at /insurer, where they open a claim with its documents, decide it with a note, and handle appeal or call-back tickets. The officer's decision flows back into the customer's chat. The agent cannot approve or deny a claim because no such tool exists. The sandbox also has a kill switch that stops every agent action. All sandbox data is synthetic.

3.9 Environment configuration and judge access

The example below is taken from .env.example. Secret values are replaced with placeholders.

.env.example

```
# Backend (clauseguard/.env)
GEMINI_API_KEY=<Gemini key>
GEMINI_API_KEYS=<extra keys, comma separated, optional>
CG_MODEL_EXTRACT=gemini-3.6-flash
CG_MODEL_FAST=gemini-3.5-flash-lite
CG_MODEL_CLAIM_DOC=gemini-3.5-flash-lite
CG_MODEL_EMBED=gemini-embedding-001
CG_CLAIM_AMOUNT_CAP_IDR=100000000
CG_ADMIN_TOKEN=<random token for the insurer desk>
CG_AUTH=supabase
SUPABASE_URL=https://<project-ref>.supabase.co
SUPABASE_PUBLISHABLE_KEY=<Supabase publishable key>
CG_DEMO_ACCOUNTS=rizky@demo.clauseguard.id=rizky,santoso@demo.clauseguard.id=santoso,dewi@demo.clauseguard.id=dewi
CG_STORAGE=postgres
DATABASE_URL=postgresql://<user>:<password>@<host>:5432/postgres

# Web (clauseguard-web/.env.local)
BACKEND_URL=https://<api-host>
NEXT_PUBLIC_APP_MODE=demo
NEXT_PUBLIC_SUPABASE_URL=https://<project-ref>.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=<Supabase publishable key>
```

Table 11 shows how judges can access the sandbox.

Table 11 Judge access

NEED	HOW
Customer app	https://clauseguard.up.railway.app
Demo accounts	rizky, santoso and dewi on the demo.clauseguard.id domain; each account loads that persona's policies and claims
Demo account password	Provided in the registration form

NEED	HOW
Insurer desk	/insurer on the same app; the officer enters the admin token
Admin token	Provided in the registration form

To try the sandbox:

1. Sign in as rizky and file a claim through the chat until the Confirm card appears.
2. Open /insurer with the admin token and decide that claim.
3. Return to the customer chat and read the explanation of the decision and the next step.

3.10 Project structure

The repository has two applications and a submission folder. The table below gives a general map; file-level detail is in the README on GitHub.

Table 12 maps the main folders of the repository.

Table 12 Repository layout

FOLDER	CONTENTS
`clauseguard/`	Python backend: rules engine, agents, sandbox insurer and API (FastAPI)
`clauseguard/clauseguard/agent/`	Agent loop and the five AI agents (conversation, fact, document reader, explainer, watchdog)
`clauseguard/clauseguard/solver.py, allocation.py, recourse.py, timeline.py`	Rules engine: coverage verdict, claim allocation across payers, counterfactual recourse and the entitlement timeline
`clauseguard/clauseguard/extractor/, plans/`	Policy-to-rule extraction and plan selection under a budget
`clauseguard/clauseguard/security/, privacy/`	Guardian enforcement point, tool registry, nonces, output guard and local redaction of personal data
`clauseguard/clauseguard/sandbox/, storage/`	Sandbox insurer API and claims desk; SQLite and Postgres storage with the hash-chained audit log
`clauseguard/data/`	Personas, sandbox policies, scripted tasks, adversarial cases and benchmark data
`clauseguard/bench/`	Evaluation harness: benchmark, paraphrase, clarification, allocation, recourse, cost and ICD tests
`clauseguard/scripts/`	Command-line tools: run tasks, run attacks, verify the audit log, fetch data, build the submission
`clauseguard/tests/`	Pytest suite (532 tests)
`clauseguard/deploy/`	Deployment notes and database migrations
`clauseguard-web/app/`	Next.js routes: landing page, customer chat (/chat) and insurer desk (/insurer)
`clauseguard-web/components/`	Chat UI, evidence cards, claim form, confirm card and landing sections

FOLDER	CONTENTS
`clauseguard-web/lib/`	API client, message parsing, personas, quick replies, PDF export and shared types
`SUBMISSION/`	Submission documents and the evidence set (reports and raw runs)

To run locally: install the backend with pip install -r requirements.txt and start it with python scripts/serve.py from clauseguard/; install the web app with npm install and start it with npm run dev from clauseguard-web/. Both read the environment variables shown in Section 3.9.

04 Core AI Agent

Five narrow agents, a deterministic engine, and code that checks every output.

4.1 AI agents and prompt design

ClauseGuard uses five AI agents. Each one performs a single narrow task and fills a closed output schema, and code validates every output before it is used. Message and document content is always treated as data, so instructions hidden inside it are ignored.

Table 13 details the task, input, prompt core, output and safeguard of each agent.

Table 13 AI agents and prompt design

AGENT	TASK	INPUT	PROMPT CORE	OUTPUT	SAFEGUARD
Conversation Agent	Reads how the message relates to the chat, then its intent, topic and tools	Message, typed situation, known facts, the agent's last question, last four exchanges	Classifies the message (answer, correction, follow-up, new situation, what-if, general), writes a short reading, then picks an intent from a fixed list and the read tools needed, with examples in two languages	Relation type, message language, facts still valid, intent, topic, tools and an action-request flag	Safety class is decided by code first; it can only drop facts; actions are accepted only on a clear request or acceptance of an offer
Fact Agent	Turns sentences into typed facts	Customer message	Fills a field only when the text states it; never guesses dates, ages, amounts or causes	Facts from a fixed vocabulary, e.g. admission date, days in hospital, bill, cause	Event facts are locked and cannot be changed to fit a policy
Document Reader Agent	Reads policies and claim documents, and lays out the insurer's claim form	Document text with personal data removed; claim clauses from the policy	Isolated component that copies document content into a schema without following instructions inside it, and designs the form's sections and fields in the insurer's terms	Typed policy rules, a claim-document summary (type, dates, total, diagnosis), or a form structure	No tools and no chat history; command-like text is flagged; form values are filled by code, never by the model
Explainer Agent	Writes the answer	Rules-engine result, last four exchanges, six example answers	Writes short reasoning notes, then two friendly, concise candidates in the customer's language	Candidate answers and the best pick	Every number, date and id is checked against the engine; clause quotes must match exactly
Watchdog Agent	Monitors deadlines and insurer decisions	Claim status, officer notes, claim facts	Classifies an officer note as a fixable defect, a reason that contradicts the policy, a valid reason, or unclear	Note type and next step	Sends nothing; appeals and resubmissions wait for the customer's confirmation

All model calls use temperature zero. If the model is unavailable, the customer receives a clear notice and can use their own API key.

4.2 Planning and tool calling

Planning runs in two layers. Code decides the safety classification first, then the Conversation Agent reads the intent of the message in the context of the conversation. Every tool is registered with a permission tier, argument types and a result type. Tool results are stored as typed handles carrying a trust label. Tier-2 tools accept only handles, so text from a document or a model can never become an argument to a submission. When several policies could receive a claim, the agent asks which one to file first.

4.3 Memory

The agent uses three kinds of memory:

1. Session memory holds facts, dates, documents read and forms awaiting confirmation.
2. Conversation memory keeps the last four exchanges to resolve references such as “that one” or “the other policy”.
3. Account history stores conversations in Postgres so they can be opened from another device.

4.4 Retrieval

Policy information is retrieved from a typed clause store. Each policy is compiled into rules such as covers, excludes, waiting period, filing deadline, pays and required documents. The rules engine retrieves the relevant rules by event, cause and insured person. This approach was chosen because insurance decisions need the exact rule together with its numbers and time windows. For diagnoses, the agent uses BM25 retrieval over 74,879 ICD-10 codes; the model picks one code from the candidate list, and code then verifies that choice.

4.5 Claim allocation model

For a single hospital bill, ClauseLens layers the payers according to Indonesian coordination-of-benefit rules:

1. Jasa Raharja pays first for road accidents involving two or more vehicles.
2. BPJS Kesehatan pays when the hospital is a partner and membership is active.
3. Employer insurance pays before private insurance.
4. Hospital cash benefits are paid alongside, because they stack.

> **Σ** Each policy pays min(policy limit, amount claimed – member cost share). Each payer receives only the unpaid remainder, so total payment never exceeds the bill. Within a layer, every filing order is evaluated and the order with the largest recovery is chosen.

4.6 Claim risk assessment

Before a claim is submitted, ClaimFlow assesses its readiness the way an insurer's officer would. Every clause that could deny the claim is checked one by one, and documents are cross-checked, for example dates against days in hospital and the receipt total against its line items. The result is a

readiness score with fixable reasons. If a claim is denied, the engine searches for the smallest legitimate change that would make it covered, such as completing a document or using another policy. Event facts are locked in code and are never altered.

4.7 Guardrails and explainability

Every answer can be traced. The “How was this decided” panel shows how the message was understood, the facts used, the clauses that applied, the engine's verdict, the sentence-check result and the audit record number. The answer checker ensures every number and date comes from the engine. If the first candidate fails, the second is checked, and the template text is used as a fallback.

05 Security Design

The AI can propose. Only the customer can act.

5.1 Permission tiers

Every tool belongs to one of four tiers, enforced in code by Guardian (security/pep.py). The autonomy switch (Observe, Draft, Act) can only narrow what the tiers already allow.

Table 14 lists the four tiers with example tools and the rule each tier enforces.

Table 14 Permission tiers

TIER	EXAMPLE TOOLS	RULE
T0 Read own data	`get_my_policies` `get_policy_clause` `get_claim_status` `get_payment_status`	Ids checked against the customer's own set; runs automatically
T1 Compute or draft	`check_coverage` `claim_readiness` `cob_plan` `draft_claim` `estimate_payout` `extract_document`	No side effects outside the session; results labelled as tool output
T2 Submit	`submit_claim` `attach_document` `schedule_callback` `file_appeal`	Only after the customer's tap: single-use server nonce, handle arguments only, at most 1 submission and 3 T2 calls per session
T3 Money or identity	`create_account_change_request (ticket only)`	Never executed by the agent; input only from a form the customer types; a person handles it

5.2 Risk controls

1. **Transaction limits.** A cap of Rp 100,000,000 per claim, at most three T2 actions and 60 tool calls per session, one T3 ticket per 24 hours, and the session stops automatically after three permission denials.
2. **Audit log.** Every message, plan, tool call, permission decision and answer is recorded in a hash chain, so a change to any record is detected during verification.
3. **Human in the loop.** The customer confirms every submission, an insurer officer decides every claim, and the insurer holds a kill switch that stops all agent actions.
4. **Data privacy.** National ID numbers, bank account numbers, card numbers, phone numbers, dates of birth and personal names are removed locally before a document is sent to the model. Every account is isolated.

5 **Regulatory compliance.** Every answer is marked as indicative. After a denial, the agent points to the official complaint channels (the insurer's complaint unit, OJK Contact 157 and LAPS SJK). Identity verification (KYC) for sensitive changes is done by the insurer through a separate channel, and the agent never moves money, so AML checks remain with the insurer.

06 Innovation & Interaction

Insurance treated as a rule program, explained in everyday language.

6.1 Difference from existing solutions

Section 1.4 compares ClauseGuard with available products. The difference lies in the core method: ClauseGuard treats an insurance policy as a rule program. The language model understands the conversation and the documents, the rules engine computes the customer's entitlement, and every answer can be proven with a clause.

6.2 Interaction design

1. **Everyday language.** Customers can write in Indonesian, English or a mix, including abbreviations and typos.
2. **One question.** The agent asks only when the answer would change the verdict, with quick-reply buttons.
3. **Evidence cards.** The verdict, clause quote, payer order, claim readiness and document contents appear as expandable cards.
4. **Correctable form.** The claim form is shown before sending, and every field can be edited.
5. **Confirm card.** Submission needs one Confirm tap that lists attached and missing documents.
6. **Proactive updates.** The insurer's decision appears in the chat with an explanation and the next step.
7. **Autonomy mode.** The Observe, Draft and Act switch is always visible at the top of the screen.

6.3 Interface design and visual hierarchy

The interface uses a single blue palette with three status colours, each with a light and a dark variant.

Table 15 lists the colours with their light and dark variants.

Table 15 Colour palette

ROLE	LIGHT MODE	DARK MODE	USE
Primary	`#1C58E3`	`#649AF7`	Main buttons, links and focus
Page background	`#FBFCFD`	`#0E0D1C`	Main background
Card	`#FFFFFF`	`#161528`	Cards and replies
Main text	`#141A29`	`#ECECF4`	Headings and body
Secondary text	`#677083`	`#9C9CB4`	Captions and small labels
Soft accent	`#E7ECF9`	`#24233E`	Highlighted backgrounds
Border	`#E1E4EA`	`#2C2B40`	Card dividers

ROLE	LIGHT MODE	DARK MODE	USE
Success	#248967	#39C697	Positive status, e.g. covered
Warning	#E9890C	#F6A831	Needs attention, e.g. deadline close
Danger	#D62940	#E75569	Negative status, e.g. denied
Navy	#0E182F	#080812	Brand icon background

Body text uses Inter and headings use Sora at weights 600 to 800. Cards have 1 rem rounded corners with a thin shadow. **Table 16** shows the order of importance of the elements on the chat screen, from top to bottom.

Table 16 Interface hierarchy

LEVEL	AREA	CONTENTS	REASON
1	Top bar and session controls	Autonomy mode, theme, persona switch, AI status and history	Always visible so the customer knows what the agent may do
2	Proactive updates	Claim-stage strip, deadline card and warning banners	Urgent items appear before the conversation
3	Conversation	Message bubbles and reply text	The main interaction path
4	Evidence cards	Verdict, clause quote, payer order, timeline, claim readiness, policy bundle and what-if	Evidence can be opened without filling the screen
5	Action cards	Claim-form preview, Confirm card and document-reading results	Actions with consequences need full attention
6	Deeper explanation	“How was this decided” panel and evidence chain	Available to those who want to inspect
7	Support panels	History, policy list and insurer desk	Opened when needed

The message box sits at the bottom with a single paper-clip icon for attaching files. Files appear as attachment cards and are sent only when the customer presses Send.

07 Testing & Results

Every figure is reported with its sample size.

Table 17 summarises the main results. Full details and reproduction steps are in the Execution Evidence Index. The full test suite (python -m pytest -q) passes 532 tests with 1 skipped, without an API key.

33 / 33
claim tasks passed

84.6%
coverage-verdict accuracy end to end, n = 390

0 / 27
attacks succeeded

92.2%
diagnosis-code accuracy, 59 of 64 lines

Table 17 Summary of results

ASPECT	RESULT	NOTES
Claim task completion	33 of 33 tasks passed	Automatic checks on verdict, amount, deadline, denial code and action
Coverage-verdict accuracy, end to end	84.6% on 390 labels, with rules ClauseGuard extracted itself	Fabricated denials 1.4%. This is the figure comparable to the language models, which read the contracts themselves
Rules engine alone	91.8% with the benchmark's own rules, 342 labels	Fabricated denials 2.9%. An upper bound for the engine, not comparable to the models
Diagnosis-code accuracy	92.2% (59 of 64 coded lines, development set of 76)	An alias dictionary alone reaches 17.2% (11 of 64); the held-out set is not yet measured
Action safety	0 of 27 attacks succeeded	Includes injection through documents
Conversation quality	42 of 47 turns passed	Everyday language, typos, corrections and what-ifs
Model cost	0.98 calls per turn	About 17% of the tokens of a baseline that reads every document
Speed without model	Median 0.18 s, p90 0.30 s	20 turns, measured locally on the rules-engine path
Speed with model	Median 8.59 s, p90 12.74 s	5 turns, measured locally; longest 40.51 s when a key rotated on quota

Speed with the model depends on the Gemini service and its quota. These measurements are small and were taken on a single machine.

08 Limitations & Known Risks

What remains open, and how each risk is handled.

This list matches the risk register in the Security Self-Assessment; the K numbers refer to that document.

Table 18 lists each known risk, its residual level and how it is handled.

Table 18 Limitations and known risks

NO.	RISK	RESIDUAL	HANDLING
K1	The Document Reader Agent can misread values; extracted rules score 7 points below the benchmark's reference rules	High	Cross-checks, an unverified label, and the customer reviews values before sending
K2	Advice text can be wrong; the permission layer limits actions but does not check whether advice is correct	Medium	Verdicts come from the rules engine, a number checker and an indicative label
K3	Text drawn inside images is not yet detected (0 of 20)	Medium	Typed output and an untrusted label block T2
K4	The audit log can be rebuilt by a party with write access to storage	Medium	Chain verification shows changes to existing records
K8	A customer-uploaded policy can mislead a verdict if its content is manipulated	Medium	Unverified label; cannot trigger T2
K9	Confirmation relies on the customer reading the card	Medium	A summary with trust labels and a per-call token
K12	Dependence on the model service and its quota	Low	Key rotation, a notice to the customer, and a bring-your-own-key option
K15	The test sets are small and written by the team	High	Every result is reported with its sample size
K18	Session memory ends when the server restarts	Low	Chat history is stored, and the customer is asked to restate the situation

Other scope limits:

1. The public benchmark covers life insurance only; Indonesian products are tested on manually verified sandbox policies.
2. 24 of the 58 benchmark scenarios, such as dividends and estate planning, are outside the scope of the rules engine.
3. BPJS Kesehatan and Jasa Raharja rules are modelled from public regulation; INA-CBG tariff grouping is not yet implemented.
4. A submitted claim cannot be edited directly; a fixable denial is followed up with a new submission after the customer confirms.

09 References

[1] KFF (2025). HealthCare.gov insurers denied nearly 1 in 5 in-network claims in 2023. kff.org, 27 January 2025.

[2] OJK (2023). Kesulitan klaim menjadi pengaduan asuransi terbanyak [Claim difficulties are the most frequent insurance complaint]. Statement of 4 December 2023, reported by Bisnis.com. finansial.bisnis.com/read/20231206/215/1721354. [In Indonesian]

[3] Kontan (2019). Asuransi Cakrawala Proteksi Indonesia luncurkan asisten virtual chat Hummi [Virtual chat assistant launch]. keuangan.kontan.co.id. [In Indonesian]

[4] Bisnis.com (2017). Strategi digital Aswata luncurkan virtual assistant [Digital strategy and virtual assistant launch]. finansial.bisnis.com/read/20171208/215/716475. [In Indonesian]

[5] Bisnis.com (2020). Asuransi Tripa lengkapi layanan digital dengan virtual assistance [Digital services with virtual assistance]. finansial.bisnis.com/read/20200603/215/1247720. [In Indonesian]

[6] Detik Oto (2018). Cukup kirim pesan singkat, asuransi kendaraan bisa langsung cair [Claims paid after a short message]. oto.detik.com/berita/d-4209016. [In Indonesian]

[7] PYMNTS (2023). Lemonade CEO says insurance platform built for AI since day one. pymnts.com.

[8] CB Insights. Company comparison of PasarPolis, Qoala and Lifepal. cbinsights.com/compare/pasarpolis-vs-qoala.

[9] Mridul, Kapa and Seneviratne (2026). A benchmark for gap and overlap analysis as a test of KG task readiness. arXiv:2604.10853.

[10] Travelers Insurance (2026). Travelers launches industry-leading agentic AI claim assistant developed with OpenAI. openai.com/index/travelers.

[11] Allianz (2025). AI at Allianz: smarter claims management, smoother settlements. allianz.com, 5 February 2025.

[12] AI Magazine (2023). Lemonade sets world record with 2-second AI insurance claim. aimagazine.com.

[13] Indotelko (2020). Qoala & Kata.AI siap dukung Strategi Nasional Kecerdasan Artifisial [Qoala and Kata.AI ready to support the National AI Strategy]. indotelko.com, 12 August 2020. [In Indonesian]

[14] Na, Kim and Cho (2022). Insurance question answering via single-turn dialogue modeling. Proceedings of COLING 2022, CAI2 Workshop, pp. 35–41.

[15] Ding et al. (2025). InsQABench: benchmarking Chinese insurance domain question answering with large language models. arXiv:2501.10943.

[16] Chen et al. (2025). INSEva: a comprehensive Chinese benchmark for large language models in insurance. arXiv:2509.04455.

[17] Beauchemin and Khoury (2026). Benchmarking large language models for Quebec insurance: from closed-book to retrieval-augmented generation. arXiv:2603.07825.

[18] Pokharel, Hassanzadeh and Agrawal (2026). From policy to logic for efficient and interpretable coverage assessment. arXiv:2601.01266.

[19] Debenedetti et al. (2025). Defeating prompt injections by design. arXiv:2503.18813.

[20] Wachter, Mittelstadt and Russell (2017). Counterfactual explanations without opening the black box.

[21] OWASP (2026). OWASP Top 10 for Agentic Applications.